

INFOB2DA | Practical Assignment 2 | Si-8

Irvan Ilahibaks 8081847 - Timon Baris 2173182 - Waling van Dongen 1176226

TA: Sita Newer Date: 30-09-2024

Task 1: Get dataset on screen

1.1 Explore the dataset as you did in the first assignment, so you have a good understanding of its content and features. Come up with an interesting one-minute story about the dataset.

When first retrieving the data, we looked at the content with different functions:

```
1 #2.0 Preprocessing
2
3 import pandas as pd
4 osiData = pd.read_csv('online_shoppers_intention.csv') #Initialize data
5 osiData.info() #viewing Data Types
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12330 entries, 0 to 12329
Data columns (total 18 columns):
#   Column                               Non-Null Count  Dtype
---  -
0   Administrative                       12330 non-null  int64
1   Administrative_Duration              12330 non-null  float64
2   Informational                        12330 non-null  int64
3   Informational_Duration               12330 non-null  float64
4   ProductRelated                      12330 non-null  int64
5   ProductRelated_Duration             12330 non-null  float64
6   BounceRates                         12330 non-null  float64
7   ExitRates                          12330 non-null  float64
8   PageValues                         12330 non-null  float64
9   SpecialDay                         12330 non-null  float64
10  Month                             12330 non-null  object
11  OperatingSystems                  12330 non-null  int64
12  Browser                          12330 non-null  int64
13  Region                          12330 non-null  int64
14  TrafficType                     12330 non-null  int64
15  VisitorType                     12330 non-null  object
16  Weekend                         12330 non-null  bool
17  Revenue                         12330 non-null  bool
dtypes: bool(2), float64(7), int64(7), object(2)
memory usage: 1.5+ MB
```

We used the info function to look at the different columns, amount of rows and data types.

Next we used the describe function to look at other info about the dataset:

```
1 #2.0 Preprocessing
2
3 osiData.describe() #summary statistics
```

	Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	Pa
count	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000	12330.000000
mean	2.315166	80.818611	0.503569	34.472398	31.731468	1194.746220	0.022191	0.043073	0.043073
std	3.321784	176.779107	1.270156	140.749294	44.475503	1913.669288	0.048488	0.048597	0.048597
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	7.000000	184.137500	0.000000	0.014286	0.014286
50%	1.000000	7.500000	0.000000	0.000000	18.000000	598.936905	0.003112	0.025156	0.025156
75%	4.000000	93.256250	0.000000	0.000000	38.000000	1464.157214	0.016813	0.050000	0.050000
max	27.000000	3398.750000	24.000000	2549.375000	705.000000	63973.522230	0.200000	0.200000	0.200000

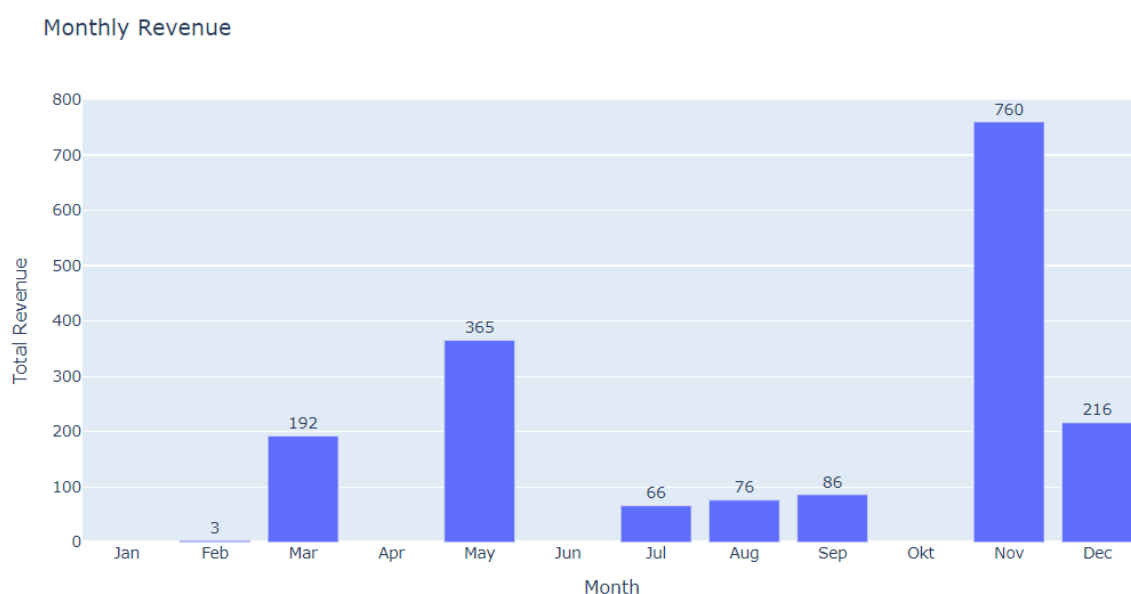
Next we retrieved as much rows as we could and showed them on screen to see

Administrative	Administrative_Duration	Informational	Informational_Duration	ProductRelated	ProductRelated_Duration	BounceRates	ExitRates	PageValues	SpecialDay	Month	Operat
2	698.500000	0	0.000000	41	1053.841565	0.002990	0.014327	0.000000	0.0	Nov	
0	0.000000	0	0.000000	2	4.000000	0.000000	0.100000	0.000000	0.0	Nov	
0	0.000000	0	0.000000	10	379.083333	0.000000	0.012500	0.000000	0.0	Dec	
0	0.000000	0	0.000000	6	223.333333	0.000000	0.008333	0.000000	0.0	May	
1	16.000000	1	29.000000	19	1083.690476	0.016000	0.024885	19.315400	0.0	May	
0	0.000000	0	0.000000	9	257.900000	0.011111	0.044444	0.000000	0.0	June	
0	0.000000	0	0.000000	21	952.533333	0.000000	0.010000	91.246000	0.0	May	
0	0.000000	0	0.000000	38	2259.200000	0.022222	0.037963	0.000000	0.2	May	
0	0.000000	0	0.000000	12	410.250000	0.000000	0.016667	0.000000	0.0	Nov	
13	408.600000	0	0.000000	21	589.421429	0.004444	0.014815	18.098572	0.0	Aug	
0	0.000000	0	0.000000	1	0.000000	0.200000	0.200000	0.000000	0.0	June	
4	70.833333	1	30.000000	27	1150.192063	0.003125	0.012500	23.554375	0.0	May	
1	7.000000	0	0.000000	11	298.000000	0.000000	0.006667	0.000000	0.0	Mar	

We explored the dataset and we decided to look into the sales in certain months. Based on the graph we made up a story to describe the data we saw.

Story

So it is the beginning of the year and the sales have slowed as people come out of the month full of festivities. January is as empty as the stockings above the fireplace. As we continue we see just a handful of sales in February, but then we finally hit March. We see an uptake in sales, filling up those beautiful stockings. Then before you know it there are no more sales in April. Not even rain has hit you, but you know what they say “After rain, comes sunshine”. It is May, well 365 sales and the sun is shining again. You hit one of the last dry spills in June. From there it's 3 months long of sailing the ship across a steady river. Before the sales boat stops in Oktober. Getting it all ready for the November and December storms.



```
#In the List are the months you looking to check
maandenOrder = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Okt', 'Nov', 'Dec']

filteredData = osiData.loc[ (osiData['Month'].isin(maandenOrder)) & (osiData['Revenue'] > 0) ]#This lines filters for the months in the previous
#as well as the colums where revenue is true

#The following lines select the month and revenue column, create a custom colum and than sort the data
monthRevenueData = filteredData[['Month', 'Revenue']].copy()
monthRevenueData['monthOrder'] = monthRevenueData['Month'].apply(lambda x: maandenOrder.index(x))
monthRevenueData = monthRevenueData.sort_values(by='monthOrder')

monthRevenueData = monthRevenueData.drop(columns=['monthOrder']) #after we drop the created column so it doesnt interfere.

groupedData = monthRevenueData[['Month', 'Revenue']].groupby('Month').sum() # we group the data by month and the total sum of the revenue
groupedData = groupedData.reindex(maandenOrder) #Making sure the order of the months is maintained.
groupedData = groupedData.reset_index() #making sure the data is ready for plotly so we reset the index

#HEre below we create the barchart in memory
fig = px.bar(
    groupedData,
    x='Month',
    y='Revenue',
    title='Monthly Revenue',
    labels={'Revenue': 'Total Revenue', 'Month': 'Month'},
    text='Revenue'
)

#we werent happy with how it all looked so we updated the layout
fig.update_traces(texttemplate='%{text}', textposition='outside')
fig.update_layout(yaxis_title='Total Revenue', xaxis_title='Month', xaxis_tickmode='array', xaxis_tickvals=maandenOrder)

fig.show()
```

1.2 Create a visualisation which shows the characteristics of online shoppers, who use 'browser 13' compared to online shoppers who use other browsers.

The first thing we wanted was to compare the different values of the two biggest browsers and compare them to browser 13. We used the code below to see which browsers were the most used. The biggest browsers are *browser 2* with 7961 users and *browser 1* with 2462 users.

```
1 import pandas as pd
2
3 osiData = pd.read_csv('online_shoppers_intention.csv')
4
5 browser_counts = osiData['Browser'].value_counts()
6
7 print(browser_counts)
8
9 |
10
```

```
Browser
2    7961
1    2462
4     736
5     467
6     174
10    163
8     135
3     105
13     61
7      49
12     10
11      6
9       1
```

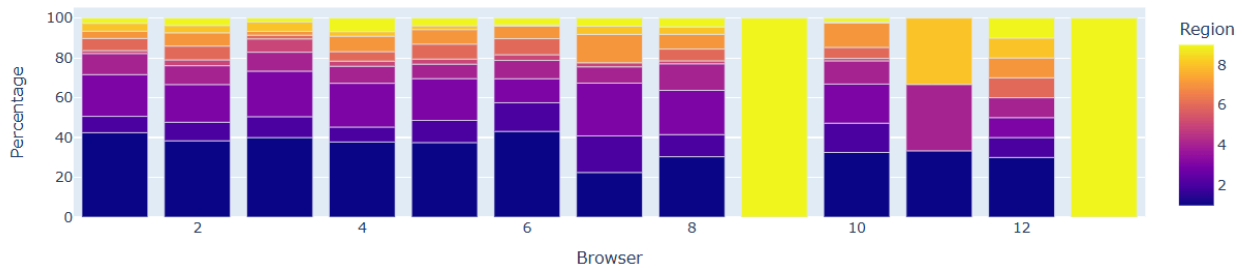
After that we looked into the data, we found some interesting things about the following values for browser 13:

- Operating Systems: Almost every user of browser 13 had operating system 8.
- Region: Everyone using browser 13 is located in region 9.
- TrafficType: Browser 13 almost only has traffic type 20.
- VisitorType: Browser 13 had almost exclusively other visitor types that did not fit in new or returning customers.

The next step was visualising the three browsers and comparing the characteristics.

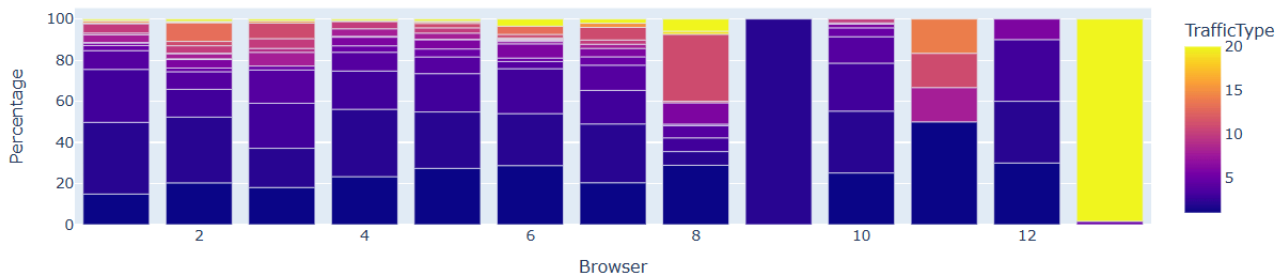
We noticed all the browsers fit in the graph, so we decided to compare browser 13 with all the other browsers. To better illustrate these findings, we visualised the characteristics of browser 13 users alongside those of other browsers. The comparative graphs showed that while all browsers fit within a similar framework, Browser 13 had unique attributes that set it apart.

Percentage Distribution of Region by Browser



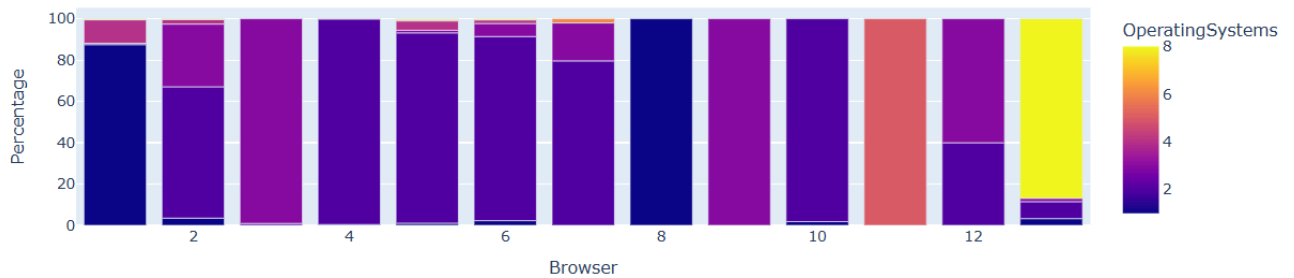
We noticed only people from region 9 used browser 13.

Percentage Distribution of Traffic Type by Browser



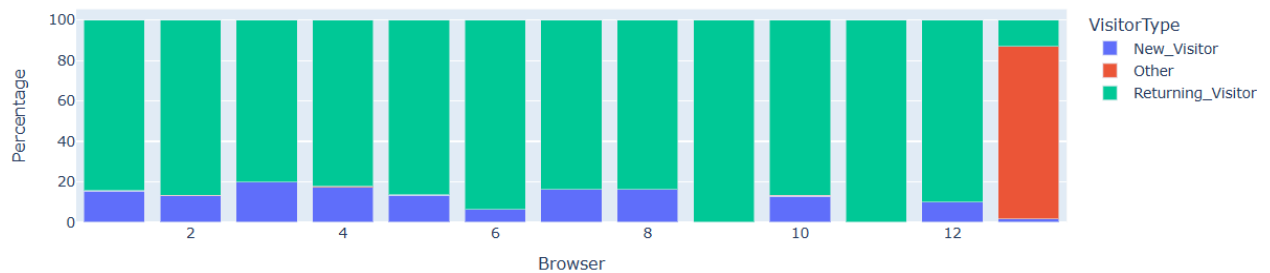
Our analysis revealed that browser 13 predominantly attracted Traffic Type 20.

Percentage Distribution of Operating Systems by Browser



We noticed mainly operating system 8 used browser 13.

Percentage Distribution of Visitor Type by Browser



Browser 13 is the only browser with mainly visitors from the “other visitor” visitor type.

Task 2: Preprocessing

Preprocessing data is a crucial step in data analysis and machine learning because raw data often contains inconsistencies, noise, and missing values. It can have different formats, scales, and irrelevant features that can affect the performance of algorithms. Preprocessing ensures that data is clean, standardised, and properly structured, allowing models to perform optimally. Preprocessing is especially crucial for clustering algorithms because these methods rely heavily on the structure and scale of the data to group similar data points.

Firstly we need to check for missing values and decide how to handle them.

```
# Check for missing values
print(osiData.isnull().sum())

# Optionally fill missing values
osiData.fillna(0, inplace=True)
```

Secondly we need to select relevant features for clustering. We think using ProductRelated, Page Values, BounceRates, Administrative_Duration, Region, OperatingSystems and Weekend are the most relevant for the algorithms.

After that we need to convert categorical variables into numerical format. For this dataset, the Weekend column can be encoded.

```
# Convert Weekend column from boolean to int (True=1, False=0)
osiData['Weekend'] = osiData['Weekend'].astype(int)
```

We normalised the features. This step is important as clustering algorithms are sensitive to the scale of the data.

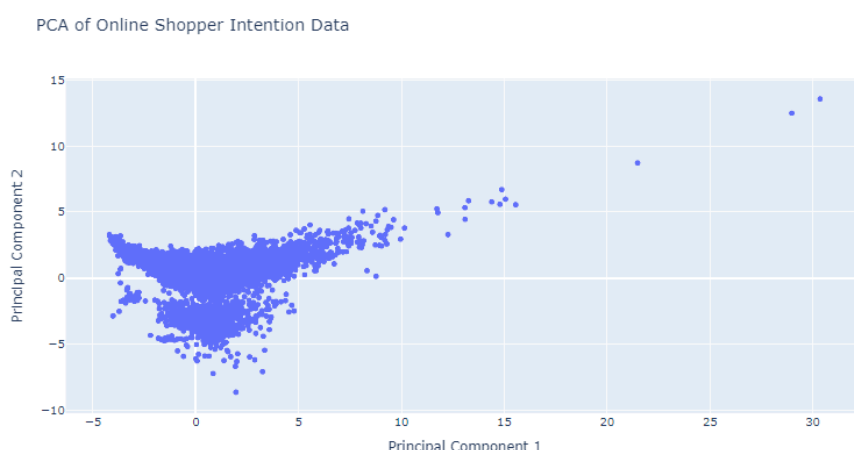
```
numeric_features = osiData.select_dtypes(include=['int64', 'float64'])

scaler = StandardScaler()
scaled_features = scaler.fit_transform(numeric_features)

scaled_data = pd.DataFrame(scaled_features, columns=numeric_features.columns)
```

We
then

used PCA on the dataset to reduce the data to only two dimensions. Two dimensions are better to show the clustering in a simple plot graph. The only downside is the loss of information. To reduce seventeen-plus dimensions to only two will lead to a lot of lost information.



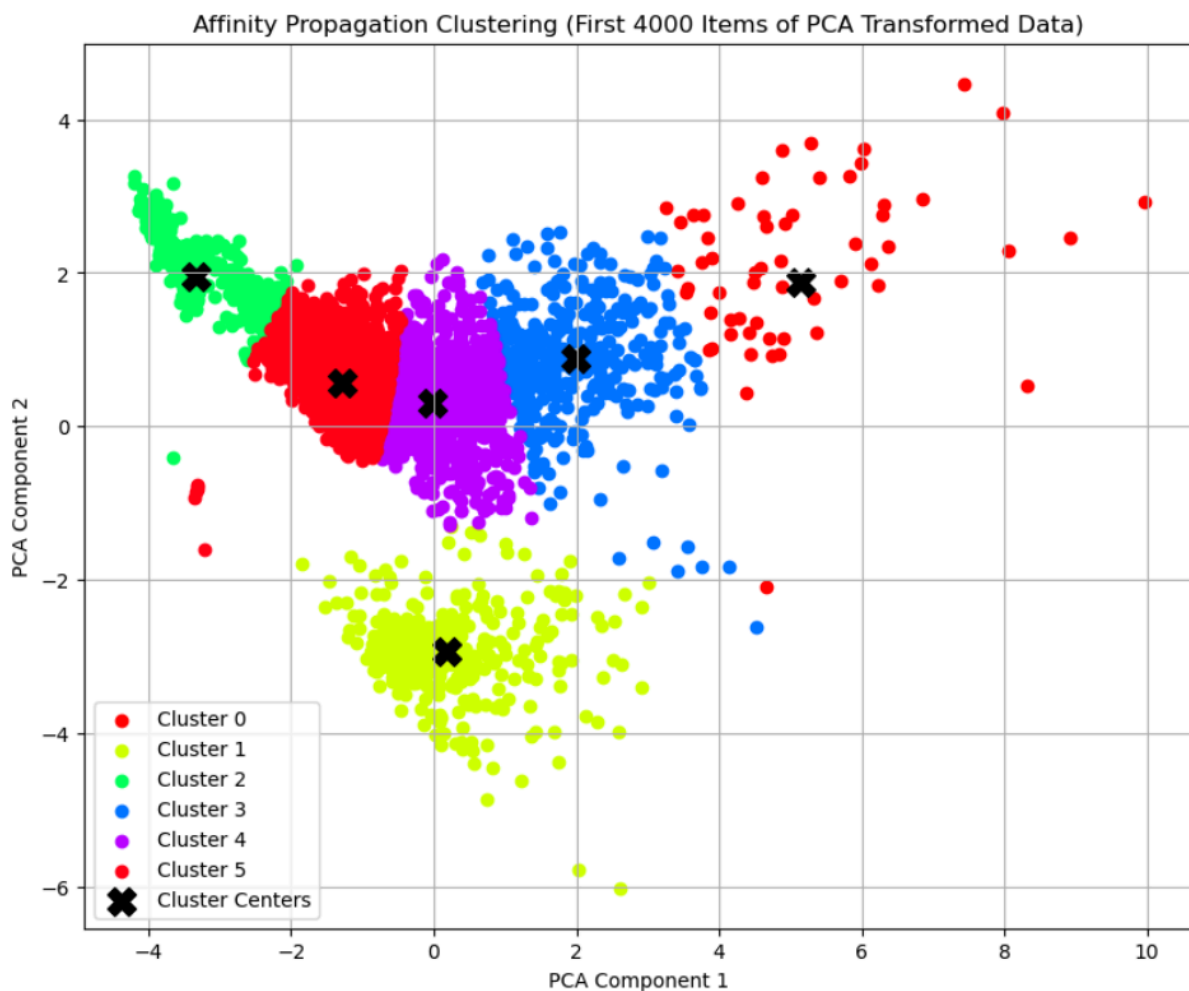
Task 3: Clustering algorithms

What is good clustering?

Good clustering consists of data points within a cluster that are close to each other, clusters should be well-separated from each other. The clusters should be consistent across different runs. The algorithm should be able to efficiently handle large datasets. Clusters should be easy to interpret and meaningful in the context of the domain.

3.1 Apply Sklearn's Affinity Propagation clustering. Visualise the created clusters.

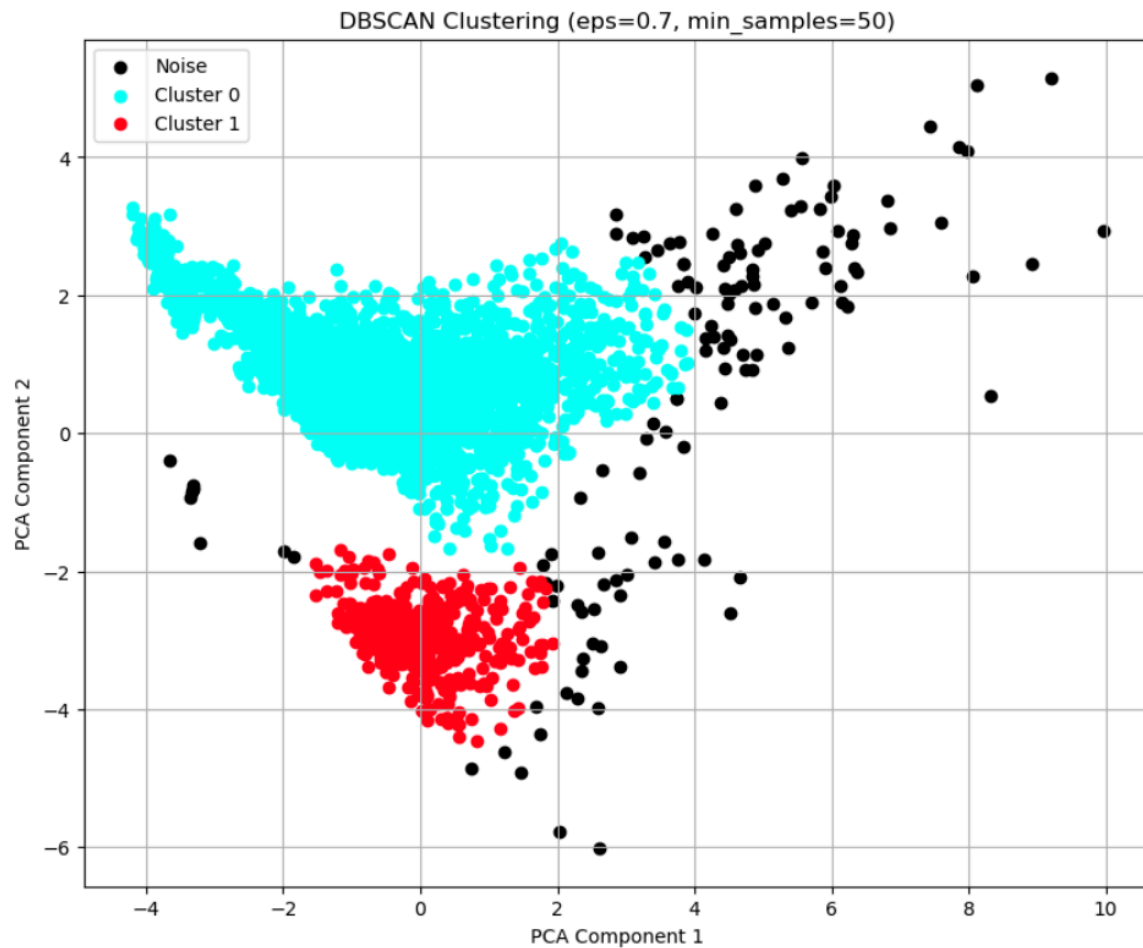
Affinity Propagation clustering is used to automatically determine the amount of clusters without previous input by the user. It uses preference to determine how likely a data point can become a cluster centre. If it is too high every data point might become a cluster centre, if it is too low only a few points become cluster centres.



3.2 Apply Sklearn's DBSCAN clustering. Visualise the created clusters.

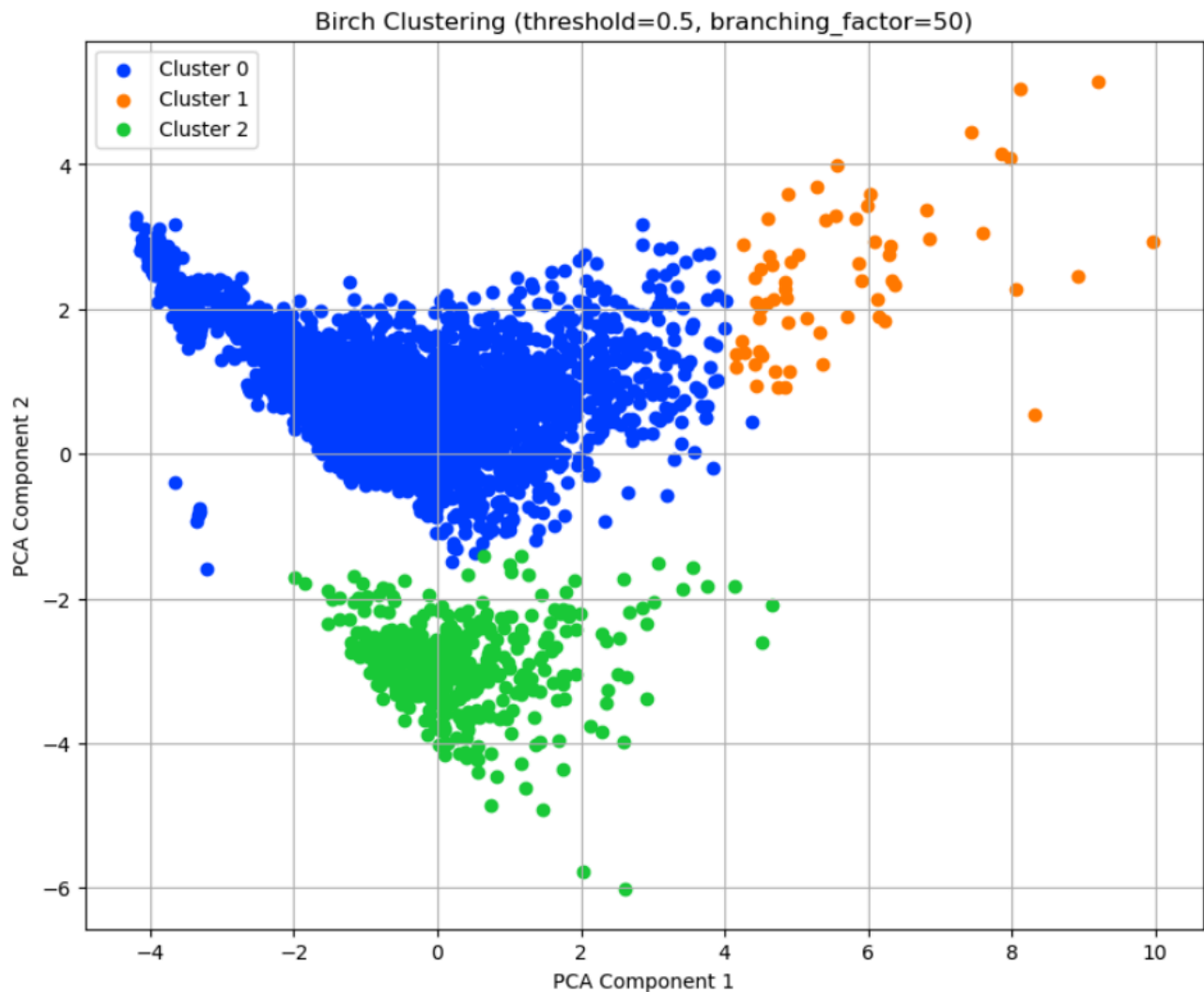
DBSCAN clustering is a clustering algorithm that identifies clusters from the density of data points. It groups points together that are close to each other while also marking points in low-density regions as outliers.

However, one of the downsides of DBSCAN is that its performance can be sensitive to the choice of parameters, specifically the radius.



3.3 Apply Sklearn's BIRCH clustering. Visualise the created clusters.

BIRCH clustering uses a hierarchical algorithm, it adds data points gradually forming larger clusters. It uses the parameter Threshold to define the radius of the cluster, the higher the number the larger the cluster. It also works more efficiently with larger datasets compared to other clustering methods.



Task 4: Evaluation of clustering methods

4.1 Manually implement the Silhouette score to evaluate the results of all clustering models.

The silhouette score is a metric used to evaluate the quality of clustering. This is done by measuring how similar a data point is to its own cluster compared to other clusters.

- For each point the score is calculated using two distances: a, the average distance to other points in the same cluster, and b the average distance to the nearest different cluster.

The silhouette score ranges from **-1 to 1**: a score close to **1** indicates the point is well-clustered, a score near **0** suggests it lies on the border between clusters, and a score of **-1** implies it may be in the wrong cluster. This score helps assess the effectiveness of clustering by indicating how well-separated the clusters are.

```
def euclideanDistance(p1, p2):
    return math.sqrt(sum((x1 - x2) ** 2 for x1, x2 in zip(p1, p2)))

def manualMean(values):
    return sum(values) / len(values) if values else 0

def silhouetteScorePoint(data, labels, i):
    point = data[i]
    label = labels[i]

    #calculate a(i) - intra-cluster distance
    sameCluster = [data[idx] for idx in range(len(data)) if labels[idx] == label]
    if len(sameCluster) == 1:
        #only one point in this cluster, silhouette score is 0
        return 0

    aI = manualMean([euclideanDistance(point, other) for other in sameCluster if other is not point])

    #Calculate b(i), meaning nearest cluster distance
    uniqueLabels = set(labels) # Use set to avoid duplicate labels
    bI = float('inf') # Set to a large number initially

    for otherLabel in uniqueLabels:
        if otherLabel != label:
            otherCluster = [data[idx] for idx in range(len(data)) if labels[idx] == otherLabel]
            avgDistance = manualMean([euclideanDistance(point, other) for other in otherCluster])
            bI = min(bI, avgDistance) # Find the minimum average distance to the nearest cluster

    #calculate the silhouette score for this point
    sI = (bI - aI) / max(aI, bI) if max(aI, bI) != 0 else 0
    return sI

# function to calculate the overall silhouette score for a cluster
def silhouetteScore(data, labels):
    scores = []
    for i in range(len(data)):
        score = silhouetteScorePoint(data, labels, i)
        scores.append(score)

    return manualMean(scores)
```

```

#usage with Affinity Propagation, DBSCAN, and BIRCH clustering
silhouetteAffinity = silhouetteScore(affDataClus, affinityLabels)
silhouetteDBSCAN = silhouetteScore(dbsClusData, dbscanLabels)
silhouetteBIRCH = silhouetteScore(birchClusData, birchLabels)

#print the silhouette scores for each clustering method
print(f'Affinity Propagation Silhouette Score: {silhouetteAffinity}')
print(f'DBSCAN Silhouette Score: {silhouetteDBSCAN}')
print(f'BIRCH Silhouette Score: {silhouetteBIRCH}')

```

```

Affinity Propagation Silhouette Score: 0.43482370396163544
DBSCAN Silhouette Score: 0.5328515643600106
BIRCH Silhouette Score: 0.5519469456834416

```

After calculating the silhouette score we can see that BIRCH and DBSCAN have good scores, BIRCH score is just a little better.

4.2 Use Sklearn's Davies Bouldin Score to evaluate the results of all clustering models.

The Davies Bouldin score can check the quality of clusters after using clustering methods. It checks how good the data has been clustered. Good clusters have similar and close points together and they should be separated from other clusters. A low score means better clusters and a higher score means a worse cluster. The score can range from zero to everything above zero.

```

from sklearn.metrics import davies_bouldin_score

#calculate Davies-Bouldin scores for Affinity Propagation, DBSCAN and BIRCH clustering methods
dbScoreAffinity = davies_bouldin_score(affDataClus, affinityLabels)
dbScoreDBSCAN = davies_bouldin_score(dbsClusData, dbscanLabels)
dbScoreBIRCH = davies_bouldin_score(birchClusData, birchLabels)

#print the Davies-Bouldin scores
print(f'Davies-Bouldin Score for Affinity Propagation: {dbScoreAffinity}')
print(f'Davies-Bouldin Score for DBSCAN: {dbScoreDBSCAN}')
print(f'Davies-Bouldin Score for BIRCH: {dbScoreBIRCH}')

```

```

Davies-Bouldin Score for Affinity Propagation: 0.7074587118843737
Davies-Bouldin Score for DBSCAN: 0.8848354869752431
Davies-Bouldin Score for BIRCH: 0.6039077791014805

```

As shown in the image, the DB-scores below 1. This indicates that the clustering is separated well. We can see that the BIRCH clustering algorithm has the best score.

4.3 Use Calinski-Harabasz Index to evaluate the results of all clustering models.

The Calinski-Harabasz index is a measure used to evaluate clustering quality by comparing the variance between clusters to the variance within clusters. A higher Calinski-Harabasz score indicates that data points within each cluster are closely grouped together (low within-cluster variance), while the clusters themselves are well-separated from one another (high between-cluster variance). A low score shows poor clustering, this means the clusters are not well separated or compact.

```
from sklearn.metrics import calinski_harabasz_score

#calculate Calinski-Harabasz scores for Affinity Propagation, DBSCAN and BIRCH clustering methods
chScoreAffinity = calinski_harabasz_score(affDataClus, affinityLabels)
chScoreDBSCAN = calinski_harabasz_score(dbsClusData, dbscanLabels)
chScoreBIRCH = calinski_harabasz_score(birchClusData, birchLabels)

#print the Calinski-Harabasz scores
print(f'Calinski-Harabasz Score for Affinity Propagation: {chScoreAffinity}')
print(f'Calinski-Harabasz Score for DBSCAN: {chScoreDBSCAN}')
print(f'Calinski-Harabasz Score for BIRCH: {chScoreBIRCH}')
```

Calinski-Harabasz Score for Affinity Propagation: 4759.164415900824
Calinski-Harabasz Score for DBSCAN: 1629.2840206163921
Calinski-Harabasz Score for BIRCH: 2885.0060095432314

After analysing the results we noticed that the Calinski-Harabasz score for affinity propagation is the highest, but all the cluster algorithms have very good Calinski Harabasz scores

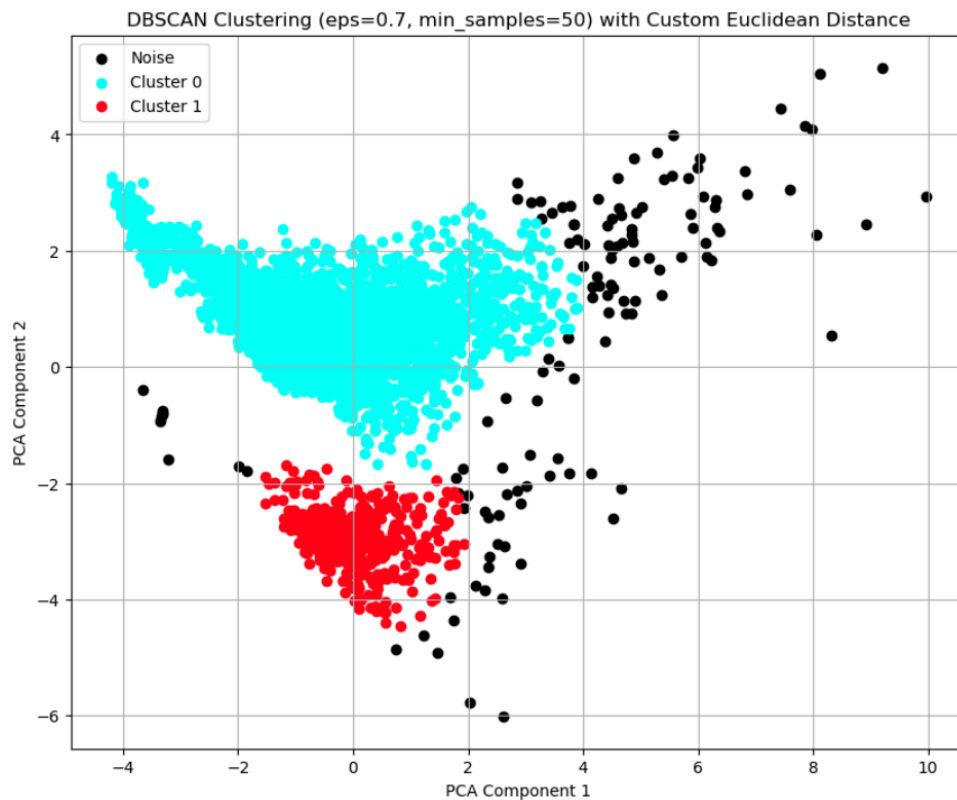
Summary

Metric	Range	Goal	Good Score
Silhouette Score	-1 to +1	Higher	Above 0,5
Davies-Bouldin Score	0 to ∞	Lower	Below 1
Calinski-Harabasz Index	0 to ∞	Higher	Above 50

Task 5: Distance functions

5.1 Manually implement a Euclidean distance function and use it as a parameter of the DBSCAN algorithm.

The Euclidean distance measures the straight line between data points. It is sensitive to large scales and outliers within the data and is suitable for normally distributed data.



#5.1 Distance Functions Manually implant euclidian

```
def manualEuclideanDistance(p1, p2):
    return sum((x1 - x2) ** 2 for x1, x2 in zip(p1, p2)) ** 0.5

dbsClusData = osiPCA[:5000, :2]

eps = 0.7
minSamples = 50

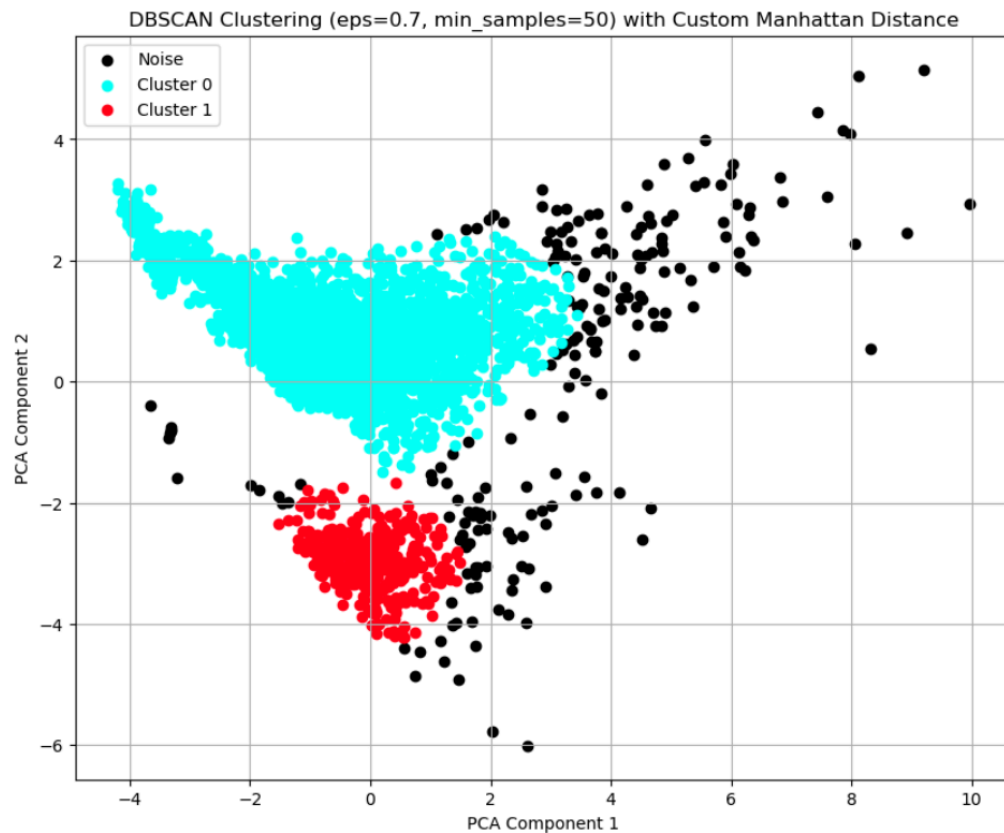
dbscan = DBSCAN(
    eps=eps,
    min_samples=minSamples,
    metric=manualEuclideanDistance
)
labelsEuclidean = dbscan.fit_predict(dbsClusData)

uniqueLabels = np.unique(labelsEuclidean)
colors = plt.colormaps['hsv'](np.linspace(0, 1, len(uniqueLabels)))

plt.figure(figsize=(10, 8))
for i, label in enumerate(uniqueLabels):
    clusterPoints = dbsClusData[labelsEuclidean == label]
    if label == -1:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color='black', label='Noise')
    else:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color=colors[i], label=f'Cluster {label}')

plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={minSamples}) with Custom Euclidean Distance')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.grid()
plt.show()
```

5.2 Manually implement a Manhattan distance function and use it as a parameter of the DBSCAN algorithm.



```
def manualManhattanDistance(p1, p2):
    return sum(abs(x1 - x2) for x1, x2 in zip(p1, p2))

dbsClusData = osiPCA[:5000, :2]

eps = 0.7
minSamples = 50

dbscan = DBSCAN(
    eps=eps,
    min_samples=minSamples,
    metric=manualManhattanDistance
)
labelsManhattan = dbscan.fit_predict(dbsClusData)

uniqueLabels = np.unique(labelsManhattan)
colors = plt.colormaps['hsv'](np.linspace(0, 1, len(uniqueLabels)))

plt.figure(figsize=(10, 8))
for i, label in enumerate(uniqueLabels):
    clusterPoints = dbsClusData[labelsManhattan == label]
    if label == -1:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color='black', label='Noise')
    else:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color=colors[i], label=f'Cluster {label}')

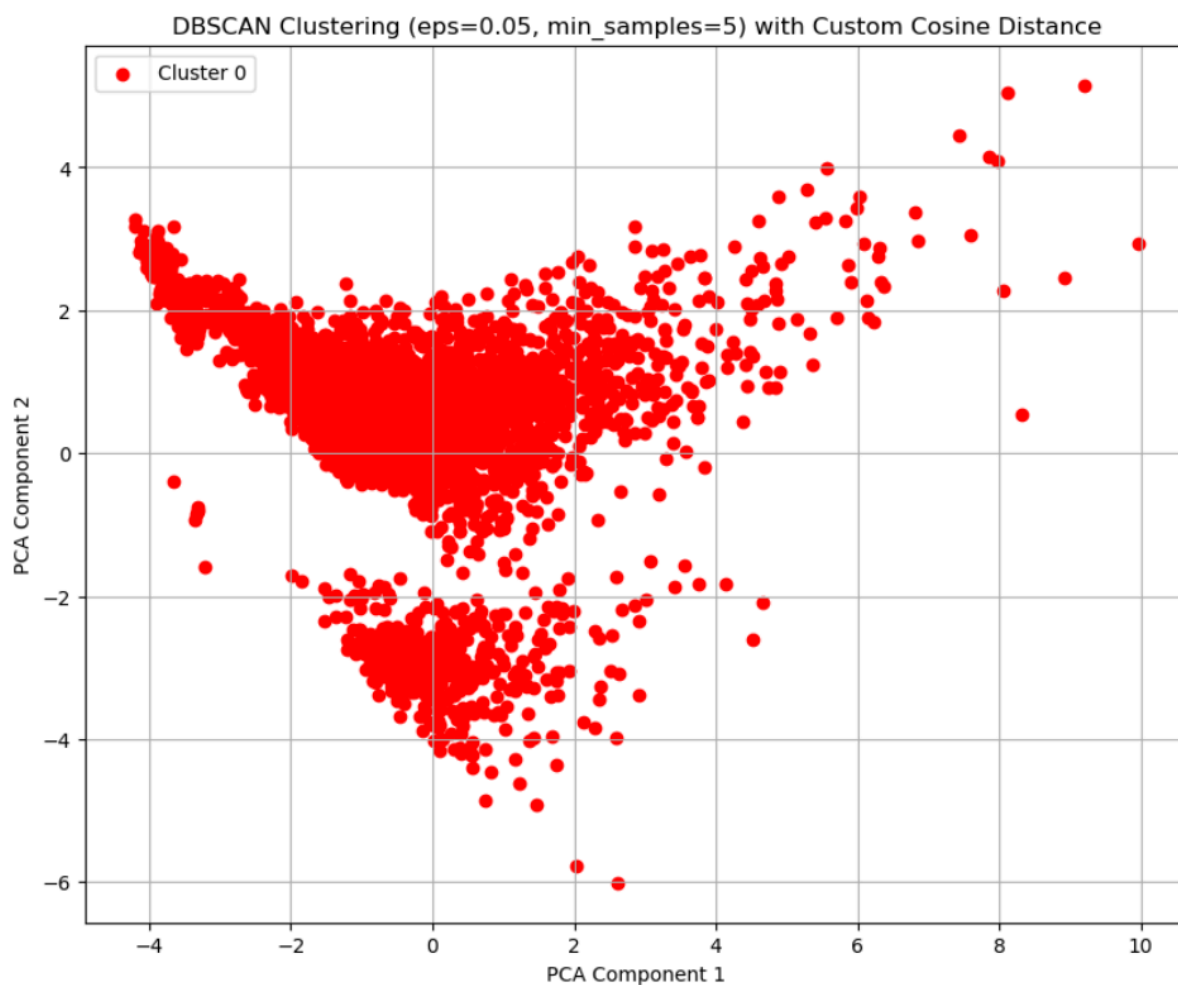
plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={minSamples}) with Custom Manhattan Distance')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.grid()
plt.show()
```

5.3 Manually implement a Cosine similarity distance function and use it as a parameter of the DBSCAN algorithm.

Cosine Distance Behavior:

- Cosine distance measures the angle between vectors, not the magnitude. In 2D (or even low-dimensional) data, many points may end up having similar directions (angles) but differing magnitudes.
- Since Cosine distance ignores magnitude, it might perceive many points as being very close (similar angles), even though they might be far apart in terms of Euclidean distance.

This is why DBSCAN, when using Cosine distance, might consider points in different regions of the space to be part of the same cluster, even if they are spread out. It essentially sees them as pointing in the same general direction.



```

def manualCosineDistance(p1, p2):
    dotProduct = sum(x1 * x2 for x1, x2 in zip(p1, p2))
    normP1 = sum(x1 ** 2 for x1 in p1) ** 0.5
    normP2 = sum(x2 ** 2 for x2 in p2) ** 0.5
    cosineSimilarity = dotProduct / (normP1 * normP2) if normP1 != 0 and normP2 != 0 else 0
    return 1 - cosineSimilarity

dbsClusData = osiPCA[:5000, :2]

eps = 0.05
minSamples = 5

dbscanCosine = DBSCAN(
    eps=eps,
    min_samples=minSamples,
    metric=manualCosineDistance
)
labelsCosine = dbscanCosine.fit_predict(dbsClusData)

uniqueLabels = np.unique(labelsCosine)
colors = plt.colormaps['hsv'](np.linspace(0, 1, len(uniqueLabels)))

plt.figure(figsize=(10, 8))
for i, label in enumerate(uniqueLabels):
    clusterPoints = dbsClusData[labelsCosine == label]
    if label == -1:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color='black', label='Noise')
    else:
        plt.scatter(clusterPoints[:, 0], clusterPoints[:, 1], color=colors[i], label=f'Cluster {label}')

plt.title(f'DBSCAN Clustering (eps={eps}, min_samples={minSamples}) with Custom Cosine Distance')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.grid()
plt.show()

```

5.4 Evaluate the effect which the different distance functions have on the DBSCAN algorithm. To evaluate, use any of the evaluation methods of the previous task. Visualise the results to support your arguments.

We decided to use the Calinski-Harabasz Index to evaluate the different distance functions. We did notice that the result of the cosine similarity distance function is only one cluster. This means that we cannot calculate a Calinski-Harabasz score for this cluster. The other clusters have scored within what we expected. The manual euclidian works exactly like the base DBSCAN while the Manhattan Distance Function took up some extra noise. Making the assumption of having a better cluster true.

```
chScoreEuclidean = calinski_harabasz_score(dbsClusData, labelsEuclidean)
chScoreManhattan = calinski_harabasz_score(dbsClusData, labelsManhattan)

print(f'Calinski-Harabasz Score for DBSCAN with Euclidean Distance: {chScoreEuclidean}')
print(f'Calinski-Harabasz Score for DBSCAN with Manhattan Distance: {chScoreManhattan}')
```

Calinski-Harabasz Score for DBSCAN with Euclidean Distance: 1629.2840206163921
 Calinski-Harabasz Score for DBSCAN with Manhattan Distance: 1681.3450109572602

```
import matplotlib.pyplot as plt

chScoreDBSCAN = calinski_harabasz_score(dbsClusData, dbscanLabels)
chScoreEuclidean = calinski_harabasz_score(dbsClusData, labelsEuclidean)
chScoreManhattan = calinski_harabasz_score(dbsClusData, labelsManhattan)
chScoreCosine = 0

chScores = [chScoreDBSCAN, chScoreEuclidean, chScoreManhattan, chScoreCosine]
labels = ['Base DBSCAN', 'Euclidean', 'Manhattan', 'Cosine (0)']

plt.figure(figsize=(8, 6))
plt.bar(labels, chScores, color=['yellow', 'blue', 'green', 'red'])

for i, score in enumerate(chScores):
    plt.text(i, score + 10, f'{score:.2f}', ha='center', va='bottom', fontsize=12)

plt.title('Calinski-Harabasz Scores for DBSCAN with Different Distance Metrics', fontsize=14)
plt.ylabel('Calinski-Harabasz Score', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

